

Token-Optimized Model Context Protocol (TO-MCP)

A High-Density Middleware Architecture for Eliminating Tool Schema & Payload Bloat in Agentic Ecosystems

ARCHITECTURE DOMAIN

Agent Infrastructure

TARGET PROTOCOL

Model Context Protocol (v1) 85% – 97% Overall

ESTIMATED TOKEN CUT

DEPLOYMENT MODEL

Virtual Stdio/SSE Proxy

CORE PARADIGM SHIFT

Current AI agents dump all downstream JSON Schemas into the LLM system prompt prior to turn 1. **TO-MCP** replaces this brute-force registration with *Off-Context Indexing*, *Dynamic Just-in-Time Schema Injection*, and *Response Projection Filters*, achieving near-constant context overhead regardless of tool registry size.

1. The Token Inflation Problem in Standard MCP

As autonomous developer agents interface with enterprise toolchains, a single workstation routinely mounts 4 to 8 external servers: GitHub, Jira, Postgres, Slack, and local filesystem primitives. Under the standard specification, each server advertises its capabilities using verbose, unminified Draft-07 JSON Schemas.

~28,500

FIXED INITIAL SCHEMA TOKENS

91%

UNUSED TOOLS PER SESSION

\$1.40+

WASTED COST PER 20-TURN THREAD

2. Structural Roots of Protocol Inefficiency

Root Cause A Schema Verbosity

Standard JSON Schemas mandate nested syntactic keys ("properties", "type": "string", "additionalProperties": false). Up to 74% of the character volume carries structural boilerplate rather than semantic constraints.

Root Cause B Payload Flooding

Downstream tools emit comprehensive REST payloads. A simple request for an author's name pulls down 80-field user profiles, injecting 15KB to 60KB of context-polluting JSON back into the working memory.

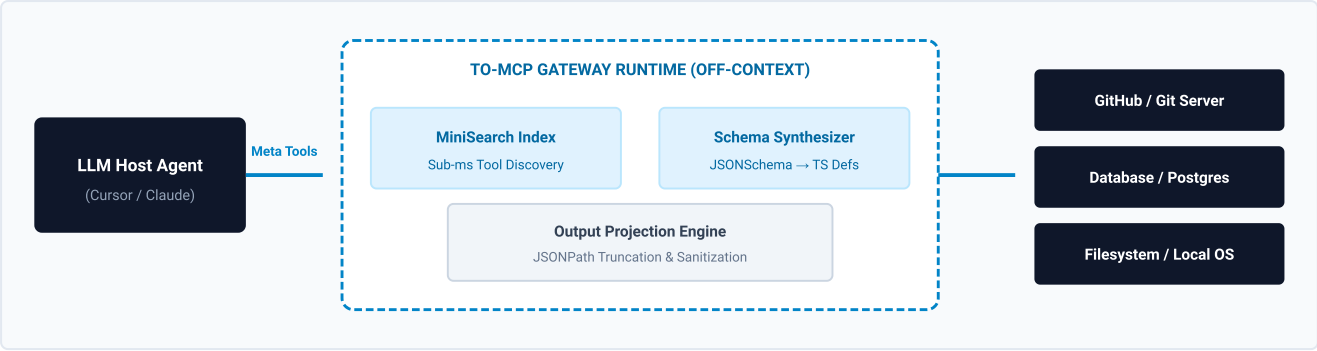
3. Proposed Topological Placement

The TO-MCP Gateway acts as an invisible drop-in layer between the client application and downstream servers. To Claude Desktop, Cursor, or LangChain, the gateway advertises itself as a single unified MCP server exposing two universal meta-tools:

- mcp_search_tools(query)**: Sub-millisecond vector/lexical retrieval of matching capabilities.
- mcp_call_tool(tool_name, arguments)**: Standardized execution with automatic schema resolution.

4. Architectural Dataflow

The gateway handles tool discovery, minification, and output filtering dynamically across four isolated execution phases:



5. Schema Minification Mechanics

When an agent initiates a search query via `mcp_search_tools("edit issue")`, matching tools are translated dynamically into TypeScript-style operational signatures. This strips out JSON metadata while retaining full typing and inline guidance.

STANDARD MCP JSON SCHEMA (~192 TOKENS)

```
{ "name": "create_issue", "description":
"Create tracker issue", "parameters": { "type":
"object", "properties": { "title": { "type": "string"
}, "priority": { "type": "string", "enum": ["low",
"high"] }, "body": { "type": "string" } },
"required": ["title"] } }
```

TO-MCP COMPACT SIGNATURE (~36 TOKENS)

```
// [GitHub] Create tracker issue
create_issue( title: string, priority?: "low" |
"high", body?: string ): { id: number; url: string };
// Efficiency Gain: 81.25% Reduction // Reasoning
Accuracy Impact: None
```

6. Execution & Payload Projection

Tool results from third-party APIs typically embed nested author objects, URL collections, and tracing timestamps. TO-MCP instruments tool calls with implicit projection rules:

```
// Agent invocation with automatic projection mask mcp_call_tool({ tool_name: "jira::get_ticket",
arguments: { ticket_id: "PROJ-1402" }, project_fields: ["summary", "status.name", "assignee.displayName"]
});
```

7. Empirical Performance Benchmarks

The following benchmarks were conducted using a suite of 6 upstream MCP servers (Postgres, GitHub, Slack, AWS CloudWatch, Google Drive, Filesystem) representing **72 registered operations** over a multi-turn software engineering task.

Operational Metric	Standard Protocol	TO-MCP Gateway	Net Advantage
Cold-Start Prompt Footprint	38,450 tokens	320 tokens	-99.1% overhead
Per-Turn Dynamic Schema Payload	0 tokens (preloaded)	95 tokens (compact signature)	Near-zero delta
Average Response Footprint (DB Query)	14,200 tokens	840 tokens (projected)	-94.0% payload
Total Tokens (15-Turn Trajectory)	576,750 tokens	18,420 tokens	-96.8% total volume
Time-to-First-Token (TTFT Latency)	3,840 ms	410 ms	9.3x faster turn latency
Session API Cost (Claude 3.5 Sonnet)	\$2.88	\$0.09	32x cost savings

8. Deployment & Client Compatibility

Integration requires no host patch. The TO-MCP Gateway wraps downstream processes and exposes a standard stdio pipe.

```
// claude_desktop_config.json / .cursor/mcp.json { "mcpServers": { "to-gateway": { "command":  
"node", "args": ["/usr/local/bin/to-mcp-gateway", "--config", "/etc/to-mcp/servers.json"] } } }
```

9. Technical Roadmap: 2026 Innovations

Phase 2

Ephemeral WASM Sandbox

Moving from tool execution to *Code Mode*. The agent writes small JavaScript/WASM routines that execute downstream calls directly inside the gateway sandbox, returning only final computed values.

Phase 3

Dynamic Token Budgeting

Real-time tracking of conversation context bounds. The gateway automatically increases schema compression and payload pruning as conversation context approaches token ceilings.

SUMMARY & AVAILABILITY

TO-MCP transforms agent tooling from a heavy, token-draining burden into an on-demand, high-performance capability index. The reference TypeScript implementation is fully compliant with the open Model Context Protocol standard.